EIN092B: Laboratorio de Procesamiento y Visualización de imágenes

Jorge Portilla, profesor

Guía de Actividades

30 de octubre de 2026

1. Objetivos

- Comprender los principios fundamentales de la visualización de datos aplicados al análisis de imágenes digitales.
- Aplicar técnicas de preprocesamiento y segmentación de imágenes para generar visualizaciones que faciliten la interpretación de estructuras relevantes en datos biomédicos.
- Utilizar bibliotecas de Python como `matplotlib`, `seaborn` y `opencv` para la representación visual de resultados derivados de operaciones matriciales y filtros de imagen.

2. Indicaciones generales.

Las actividades planteadas en esta guía deben ser entregadas durante la sesión de laboratorio.

2.1. Sobre el proceso de revisión de actividades.

Para el proceso de revisión consideren lo siguiente:

- La revisión y evaluación de cada una de las actividades se realizará en dos etapas: (i) una validación básica de requerimientos funcionales mínimos durante la sesión, y (ii) una verificación de requerimientos funcionales y no funcionales posterior a la sesión.
- Durante la sesión, avise al staff (profesor o ayudante) cuando complete una o varias actividades para que muestre y explique sus resultados, y responda las preguntas que se le realizarán sobre los conceptos asociados.
- Cualquier integrante del grupo debe ser capaz de responder las preguntas. Si las respuestas no son satisfactorias, o algún integrante no es capaz de responder, no se revisará la actividad hasta que demuestren que TODOS los integrantes del grupo demuestren un entendimiento mínimo sobre lo que están entregando.
- Durante la sesión, el staff marcará en una planilla cuando Ud. considere una actividad terminada y haya pasado la interrogación. El tener la actividad marcada como revisada en la sesión no entrega puntaje, y solo asegura que cumple con requerimientos para que entregue sus archivos y pase a la siguiente etapa de revisión. El formato de entrega de sus archivos se indica en la sección 2.2.
- En el caso de las actividades no entregadas en la sección correspondiente contará como no entregada a tiempo y **deberá entregar la actividad corregida en la siguiente sesión, aplicando descuentos por atraso**.
- En el caso de que algún integrante del grupo no este presente cuando su grupo correspondiente presente la evaluación y su correspondiente interrogación, deberá presentarlo por su propia cuenta en las siguientes sesiones y sufriendo la penalización por atraso.

2.2. Indicaciones de formato para entrega.

La evaluación constará de dos partes. La primera parte consiste en una validación básica y preguntas sobre los códigos propuestos por el grupo. La segunda parte constará de una verificación de sus códigos y respuestas, la cual se realizará sobre los archivos que deberá entregar vía Aula USM, los cuales deben seguir estrictamente las especificaciones que se le indican a continuación:

1. Debe crear una carpeta con el nombre *GXX_ Sesion11*, donde *XX* el número de grupo. Por ejemplo, la carpeta para el grupo 5 se llamaría `G5_Sesion11`. El número de su grupo fue definido en la sesión anterior.

2. Dentro de esta carpeta, debe incluirse un archivo de Jupyter Notebook que contenga el desarrollo de las actividades y las respuestas correspondientes.

Recordar que **será responsabilidad del grupo entregar de manera correcta los archivos**. Se permitirá el envío de los archivos hasta las **23:59 del 17 de octubre**. Sin embargo, solo se revisarán las actividades marcadas como revisadas durante la sesión. Cualquier actividad adicional no será considerada en la revisión.

3. Introducción

En el ámbito de la ciencia de datos, la visualización constituye una herramienta esencial para interpretar, analizar y comunicar información compleja de manera efectiva. En el caso particular del procesamiento de imágenes digitales, la visualización permite representar estructuras internas, contrastes y patrones que no son evidentes a simple vista, facilitando así la comprensión de los datos y la toma de decisiones basadas en evidencia visual.

3.1. Kernels en el procesamiento y visualización de imágenes

En el procesamiento digital de imágenes, un **kernel** (o filtro convolucional) es una pequeña matriz numérica que se utiliza para modificar la intensidad de los píxeles de una imagen. Mediante la operación de *convolución*, el kernel se desplaza sobre la imagen y realiza una combinación ponderada de los píxeles vecinos, generando una nueva imagen procesada. Esta operación puede emplearse para distintos fines como suavizado, realce de bordes, detección de contornos o eliminación de ruido.

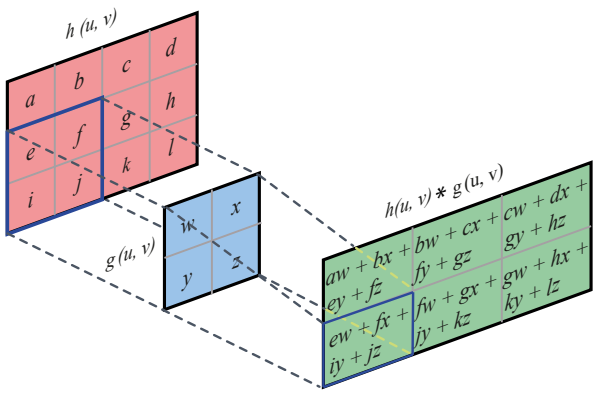


Figura 1: Ejemplo de una operación de convolución 2D.

La Figura 1 muestra un ejemplo de operación de convolución 2D, una representación adaptada de [1]. La operación básica de convolución se define como:

$$h(u,v) * g(u,v) = \sum_{i=-k}^{k} \sum_{j=-k}^{k} h(u+i, v+j) \cdot g(i,j),$$

donde $h(u,v)$ es la imagen original y $g(u,v)$ es el kernel.

A continuación, se presentan algunos de los kernels más utilizados en análisis de imágenes.

3.1.1. Filtro Laplaciano

El filtro **Laplaciano** se basa en la segunda derivada de la intensidad de la imagen y permite resaltar zonas de transición brusca, es decir, bordes. Un kernel común del operador Laplaciano es:

$$L = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Este filtro resalta los contornos donde ocurre un cambio rápido de intensidad y puede combinarse con filtros suavizadores (como el de mediana o gaussiano) para reducir el ruido antes de su aplicación.

3.1.2. Filtro Sobel

El filtro **Sobel** calcula una aproximación del gradiente de la imagen en las direcciones horizontal y vertical, permitiendo detectar la orientación de los bordes. Se utilizan dos kernels distintos:

$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad \text{y} \quad S_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

La magnitud del gradiente se calcula como:

$$G = \sqrt{S_x^2 + S_y^2}$$

Este resultado puede visualizarse como una imagen de bordes en la que los contornos aparecen más brillantes.

3.1.3. Filtro Prewitt

El filtro **Prewitt** es similar al Sobel, pero utiliza coeficientes más simples, lo que reduce la carga computacional. Sus kernels son:

$$P_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \quad \text{y} \quad P_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

El resultado también se calcula mediante la magnitud del gradiente, similar al filtro Sobel. Estos filtros son particularmente útiles para visualizar bordes horizontales, verticales y diagonales en las imágenes MRI.

3.1.4. Visualización de resultados

Una vez aplicados los filtros, las imágenes resultantes pueden visualizarse mediante mapas de intensidad, escalas de grises o representaciones con color falso, lo que facilita la comparación visual de los bordes detectados. Esto permite analizar la estructura cerebral y evaluar la eficacia de cada filtro en la detección de contornos relevantes para la segmentación.

Ejemplo en Python:

```python
import cv2
import matplotlib.pyplot as plt

# Cargar imagen en escala de grises
img = cv2.imread('brain_mri.png', cv2.IMREAD_GRAYSCALE)

# Aplicar filtro Sobel
sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
sobel = cv2.magnitude(sobelx, sobely)

# Visualizar
plt.figure(figsize=(8,4))
plt.subplot(1,2,1); plt.imshow(img, cmap='gray'); plt.title('Original')
plt.subplot(1,2,2); plt.imshow(sobel, cmap='gray'); plt.title('Bordes Sobel')
plt.show()
```

3.2. Técnicas para evitar la pérdida de información en los bordes

Durante la aplicación de filtros o kernels mediante la operación de convolución, es común enfrentar el problema de la **pérdida de información en los bordes** de la imagen. Esto ocurre porque, al superponer el kernel sobre los píxeles cercanos al borde, algunos elementos del kernel quedan fuera del dominio de la imagen, impidiendo calcular una convolución completa en esas zonas.

Para resolver este problema y conservar la integridad visual de la imagen, se utilizan diferentes técnicas de *padding* (relleno de bordes), las cuales extienden la imagen artificialmente antes de aplicar el filtro.

3.2.1. Zero Padding

La técnica más simple consiste en rellenar los bordes con ceros. Aunque mantiene el tamaño original de la imagen, puede generar un efecto de oscurecimiento en los bordes, especialmente en filtros de alta frecuencia (como los detectores de bordes).

$$\text{Imagen extendida con ceros: } \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & a & b & c & 0 \\ 0 & d & e & f & 0 \\ 0 & g & h & i & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

3.2.2. Replicación de bordes

En esta técnica, los valores de los píxeles del borde se replican hacia afuera. De esta manera, las zonas externas imitan los valores más cercanos, reduciendo la distorsión en los resultados del filtrado.

Ejemplo: $\begin{bmatrix} a & a & b & c & c \\ a & a & b & c & c \\ d & d & e & f & f \\ g & g & h & i & i \\ g & g & h & i & i \end{bmatrix}$

3.2.3. Reflejo (Reflect Padding)

Consiste en extender la imagen reflejando los valores adyacentes en el borde, lo que produce una transición más natural que el zero padding o la replicación. Es ampliamente utilizada en bibliotecas de procesamiento de imágenes como `OpenCV` o `scipy.ndimage`.

Ejemplo (reflejo horizontal): $\begin{bmatrix} c & b & a & b & c \\ f & e & d & e & f \\ i & h & g & h & i \\ f & e & d & e & f \\ c & b & a & b & c \end{bmatrix}$

3.2.4. Wrap Padding

En este caso, la imagen se trata como si fuera periódica, de modo que los bordes se "envuelven" alrededor de la imagen. Es útil en algunas aplicaciones de procesamiento de señales o transformadas, pero puede producir artefactos visuales en imágenes naturales.

Implementación práctica

En `OpenCV`, el tipo de relleno se especifica mediante el parámetro `borderType` en funciones como `cv2.filter2D()`, `cv2.GaussianBlur()` o `cv2.copyMakeBorder()`:

Ejemplo en Python:

```python
import cv2
import matplotlib.pyplot as plt

img = cv2.imread('brain_mri.png', cv2.IMREAD_GRAYSCALE)

# Aplicar un filtro Laplaciano con diferentes estrategias de borde
lap_zero = cv2.Laplacian(img, cv2.CV_64F, borderType=cv2.BORDER_CONSTANT)
lap_reflect = cv2.Laplacian(img, cv2.CV_64F, borderType=cv2.BORDER_REFLECT)

# Comparar visualmente
plt.figure(figsize=(10,4))
plt.subplot(1,3,1); plt.imshow(img, cmap='gray'); plt.title('Original')
plt.subplot(1,3,2); plt.imshow(lap_zero, cmap='gray'); plt.title('Zero Padding')
plt.subplot(1,3,3); plt.imshow(lap_reflect, cmap='gray'); plt.title('Reflect
    Padding')
plt.show()
```

3.3. Método de Otsu

El método de Otsu es una técnica automática de binarización utilizada para separar los píxeles de una imagen en dos clases: fondo y objeto. Su objetivo es encontrar un umbral óptimo t^* que maximice la diferencia estadística entre ambas clases, es decir, la *varianza entre clases*. Cuando esta varianza es máxima, se logra una mejor separación de los niveles de intensidad, produciendo una segmentación más precisa sin intervención manual.

3.3.1. Funcionamiento del método

El método se basa en el análisis del histograma de la imagen, el cual representa la frecuencia de cada nivel de intensidad. Otsu asume que la imagen contiene dos grupos de píxeles (bimodal), ver Figura 2: uno correspondiente al fondo y otro al objeto. A partir de esa distribución, el algoritmo evalúa todos los posibles umbrales t y selecciona aquel que maximiza la varianza entre las clases.

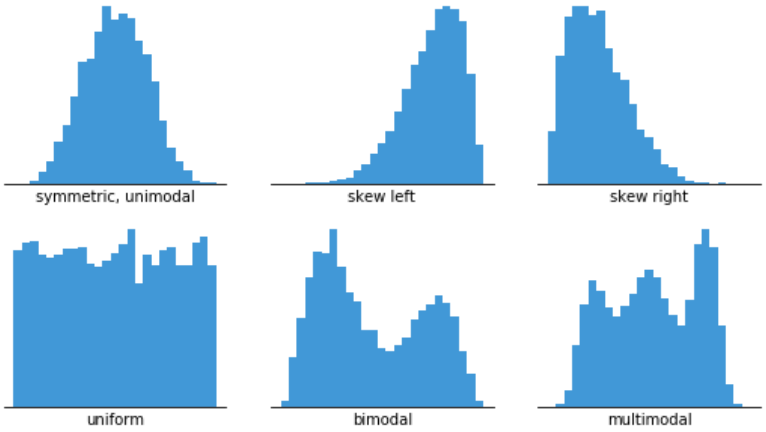


Figura 2: Diferentes tipos de histogramas.

3.3.2. Formulación matemática

Sea una imagen en escala de grises con niveles de intensidad $\{0, 1, \ldots, L-1\}$ y un histograma normalizado $p(i) = \frac{n_i}{N}$, donde:

- n_i: número de píxeles con intensidad i,
- N: número total de píxeles,
- $p(i)$: probabilidad de aparición de la intensidad i.

La suma de todas las probabilidades es igual a 1:

$$\sum_{i=0}^{L-1} p(i) = 1$$

La imagen se divide en dos clases según un umbral t:

$$C_1 = [0, t] \quad \text{(fondo)}, \qquad C_2 = [t+1, L-1] \quad \text{(objeto)}$$

Las probabilidades acumuladas de cada clase se definen como:

$$\omega_1(t) = \sum_{i=0}^{t} p(i), \qquad \omega_2(t) = \sum_{i=t+1}^{L-1} p(i)$$

donde $\omega_1(t)$ y $\omega_2(t)$ representan la proporción de píxeles pertenecientes al fondo y al objeto, respectivamente.

Las medias de cada clase (promedio de intensidades dentro de cada grupo) se calculan como:

$$\mu_1(t) = \frac{\sum_{i=0}^{t} i\, p(i)}{\omega_1(t)}, \qquad \mu_2(t) = \frac{\sum_{i=t+1}^{L-1} i\, p(i)}{\omega_2(t)}$$

La media total de la imagen, considerando todos los niveles de intensidad, es:

$$\mu_T = \sum_{i=0}^{L-1} i\, p(i)$$

Luego, se calcula la varianza entre clases, que mide cuán separadas están las intensidades promedio del fondo y del objeto:

$$\sigma_B^2(t) = \omega_1(t)[\mu_1(t) - \mu_T]^2 + \omega_2(t)[\mu_2(t) - \mu_T]^2$$

Cuanto mayor sea esta varianza, mayor será la separación entre las dos clases y, por tanto, más claro el umbral.

Finalmente, el umbral óptimo t^* se obtiene buscando el valor que maximiza la varianza entre clases:

$$t^* = \arg\max_t \sigma_B^2(t)$$

3.3.3. Interpretación

El valor t^* representa el punto de corte en el histograma donde se logra la mejor división entre el fondo (valores bajos) y el objeto (valores altos). En términos prácticos, los píxeles con intensidades menores o iguales a t^* se asignan al fondo (valor 0), mientras que los superiores se asignan al objeto (valor 255 en una imagen binaria). De este modo, el método de Otsu permite obtener una segmentación automática y objetiva, sin requerir que el usuario elija manualmente el umbral.

3.3.4. Implementación

```python
import cv2
import matplotlib.pyplot as plt

# Cargar imagen en escala de grises
img = cv2.imread('brain_mri.png', cv2.IMREAD_GRAYSCALE)

# ---- Binarizacion automatica (Otsu) ----
_, binary_otsu = cv2.threshold(img, 0, 255,
                               cv2.THRESH_BINARY + cv2.THRESH_OTSU)

plt.figure(figsize=(12,4))
plt.subplot(1,3,1); plt.imshow(img, cmap='gray'); plt.title('Original')
plt.subplot(1,3,2); plt.imshow(binary_manual, cmap='gray')
plt.title('Manual (t=100)')
plt.subplot(1,3,3); plt.imshow(binary_otsu, cmap='gray')
plt.title(f'Otsu (t={int(_):d})')
plt.show()
```

4. Actividades

Se solicita realizar procesamientos computacionales a matrices, considerando el uso de imágenes digitales de resonancia magnética (MRI). El objetivo principal es segmentar el cerebro, separándolo del resto de las estructuras, incluyendo el cráneo, mediante técnicas de filtrado, análisis de histogramas y binarización. Para esto, deberá hacer un uso correcto de las bibliotecas disponibles en Python.

A continuación, se detallan las etapas que deben desarrollarse:

4.1. Cargar y visualizar correctamente las imágenes

El primer paso consiste en la carga y visualización de las imágenes MRI en escala de grises, ver Figura 3. Este proceso permite verificar el formato, las dimensiones y las características visuales del conjunto de datos.

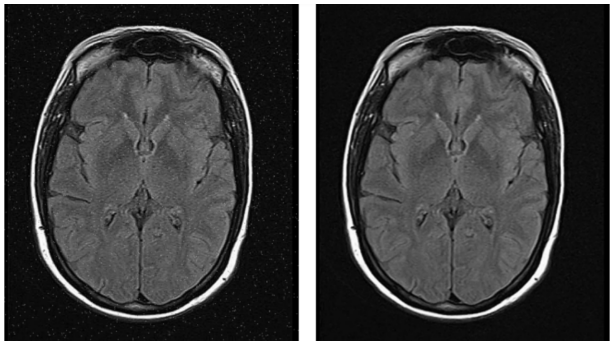


Figura 3: Ejemplo de imágenes MRI. Izquierda: imagen con ruido. Derecha: imagen filtrada.

4.2. Aplicar un filtro de mediana

A continuación, se debe aplicar un filtro de mediana a las imágenes. Este filtro asigna a cada píxel la mediana de los valores dentro de una ventana (por ejemplo, de 3×3), permitiendo eliminar el ruido impulsivo sin afectar significativamente los bordes, ver Figura 4.

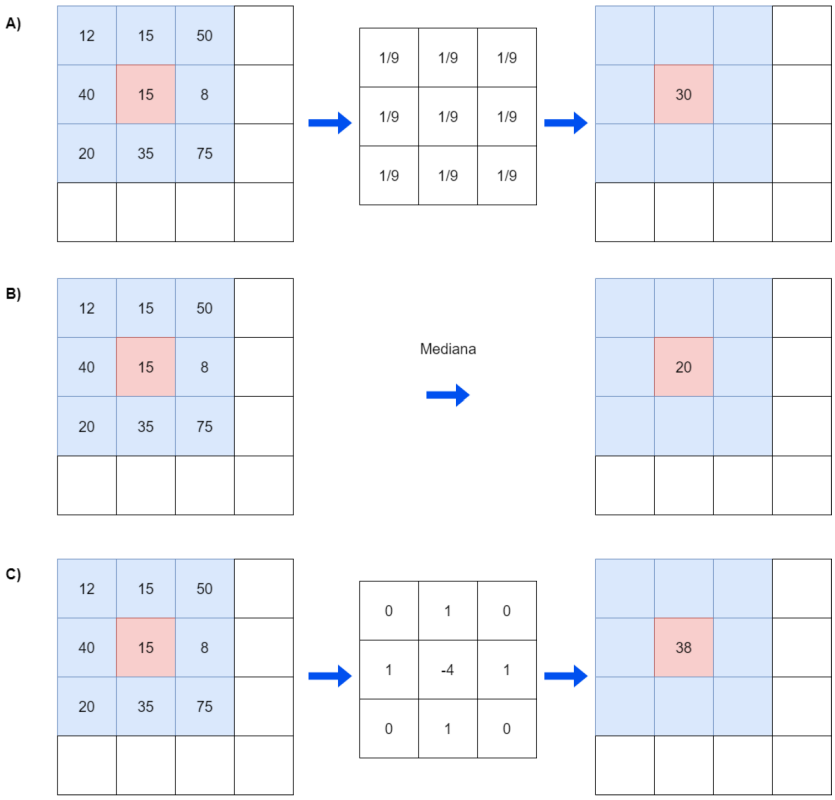


Figura 4: Ejemplo A) media. B) mediana.
C) $38 = (12\times0)+(15\times1)+(50\times0)+(40\times1)+(15\times(-4))+(8\times1)+(20\times0)+(35\times1)+(75\times0)$ (Laplaciano).

4.3. Mostrar el histograma de las imágenes

El histograma representa la distribución de los niveles de intensidad en la imagen y permite analizar la diferencia entre el cerebro y otras estructuras, ver Figura 5. A partir de este análisis, se pueden identificar los rangos de intensidad que caracterizan las regiones de interés.

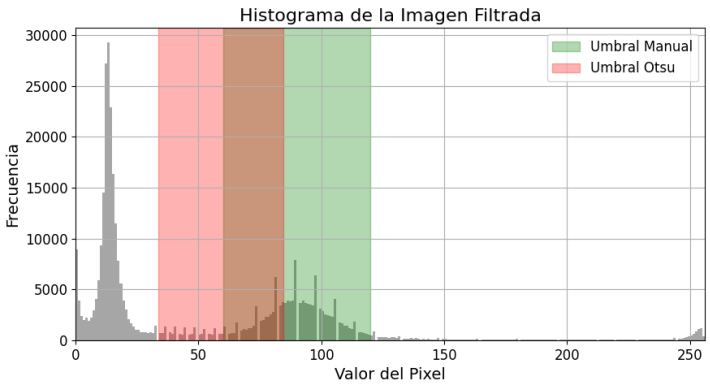


Figura 5: Ejemplo de histograma de intensidades para una imagen MRI.

4.4. Implementar un proceso de binarización

El siguiente paso consiste en crear una máscara binaria del cerebro mediante un proceso de binarización. Este puede realizarse de forma manual, seleccionando un umbral a partir del histograma, o mediante un método automático como el de **Otsu**[1].

4.5. Aplicar la máscara binaria

La máscara binaria obtenida se aplica sobre la imagen filtrada (sin ruido) para extraer únicamente el cerebro y eliminar las estructuras externas. Este paso permite obtener una representación visual clara y segmentada del área de interés, ver Figura 6.

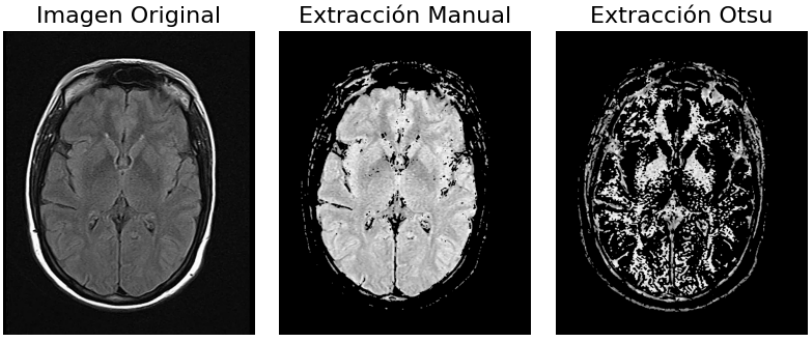


Figura 6: Ejemplo de binarización del cerebro mediante el método de Otsu.

4.6. Analizar resultados

Finalmente, se deben analizar los resultados obtenidos y evaluar la efectividad de la umbralización realizada de acuerdo con el objetivo original. Además, se debe examinar si existen diferencias en el desempeño de la umbralización según el ángulo de adquisición de la imagen, el estado del paciente u otros factores que puedan influir en la segmentación.

[1] `https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html`

Nota: En todos los casos, realice una formulación matemática de los problemas. Puede usar como ejemplo la aplicación del filtro de media en una imagen:

$$g(x,y) = \frac{1}{m \times n} \sum_{i=-a}^{a} \sum_{j=-b}^{b} f(x+i, y+j)$$

donde:

- $g(x,y)$: valor del píxel resultante en la imagen filtrada.
- $f(x+i, y+j)$: valor de intensidad de los píxeles vecinos en la imagen original.
- $m \times n$: dimensiones del kernel de filtrado (por ejemplo, 3×3).
- $a = \frac{m-1}{2}$ y $b = \frac{n-1}{2}$: indican el desplazamiento del kernel en torno al píxel central.

Bonus

Seleccione una de las siguientes opciones:

- Ajustar la umbralización para segmentar más de una estructura de la imagen, evaluando factibilidad y posibles desafíos.
- Implementar un filtro de detección de bordes basado en un operador Laplaciano, Sobel o Prewitt, justificando su decisión.
- Investigar y realizar otro procedimiento que considere el uso de cálculos básicos de álgebra lineal o cálculo diferencial.

Referencias

[1] Ian Goodfellow, Yoshua Bengio y Aaron Courville. *Deep Learning*. MIT Press, 2016.